

# EdgeSign

## Technical Setup & Execution Report

*Laptop + Raspberry Pi Architecture*

|                |                                                      |
|----------------|------------------------------------------------------|
| Platform       | Windows Laptop + Raspberry Pi                        |
| Entry Points   | laptop_rec.py (Laptop)   pi_sender.py (Raspberry Pi) |
| Communication  | WebSocket — Port 8765   Web/API Server — Port 3000   |
| Python Version | 3.10+ recommended on both devices                    |

### 1. Code Files & Where to Run

The project is split into two main code files, each running on a different device:

| File          | Device         | Role                                          |
|---------------|----------------|-----------------------------------------------|
| laptop_rec.py | Windows Laptop | Receiver / Server (WebSocket + Web API)       |
| pi_sender.py  | Raspberry Pi   | Sender (camera capture + landmark extraction) |

### 2. Software Used

#### 2.1 Laptop (Windows)

| Laptop — Windows — Required Python Packages |
|---------------------------------------------|
| Python 3.10+ (recommended)                  |
| pip (Python package installer)              |
| aiohttp                                     |
| aiohttp-cors                                |
| websockets                                  |

**Laptop — Windows — Required Python Packages**

numpy

opencv-python

easyocr

openai-whisper

google-gemai

pyttsx3

pybluez

Install all Laptop packages with a single command:

```
pip install aiohttp aiohttp-cors websockets numpy opencv-python easyocr  
openai-whisper google-gemai pyttsx3 pybluez
```

## 2.2 Raspberry Pi

**Raspberry Pi — Raspberry Pi OS — Required Python Packages**

Python 3.10+ (recommended)

pip (Python package installer)

opencv-python

mediapipe

websocket-client

numpy

sounddevice

luma.oled

pillow

RPi.GPIO

Install all Raspberry Pi packages with a single command:

```
pip install opencv-python mediapipe websocket-client numpy sounddevice  
luma.oled pillow RPi.GPIO
```

## 2.3 Notes

- Optional packages (audio, OLED, Bluetooth, OCR, Whisper, Gemini, TTS) can be skipped if those features are not required.
- Ensure the camera is connected and accessible on the Raspberry Pi before running.
- Ensure the model file exists on the Laptop at the following path:

```
models/sign_language_model.pkl
```

## 3. Pre-Run Setup

---

### 3.1 Network Configuration

Both devices must be connected to the same local network before starting.

In `pi_sender.py`, locate and update the following line with the Laptop's IP address or hostname:

```
PC_IP = "YOUR_LAPTOP_IP_OR_HOSTNAME"
```

The WebSocket URL is automatically derived from this value:

```
WS_URL = f"ws://{PC_IP}:8765"
```

### 3.2 Gemini API Key (Optional)

If the AI-powered sentence generation feature is needed, set the Gemini API key in `laptop_rec.py`:

```
GEMINI_API_KEY = "YOUR_API_KEY"
```

This step can be skipped if sentence generation is not required.

---

## 4. Execution Instructions

---

Follow these steps in the exact order shown below. The Laptop server must be started before the Raspberry Pi.

### Step 1: Start the Laptop Receiver / Server

1. Open a terminal in the project root folder on the Laptop.
2. Run the following command:  
`python laptop_rec.py`
3. This will start two services:
  - WebSocket server on port 8765
  - Web / API server on port 3000

### Step 2: Start the Raspberry Pi Sender

1. Open a terminal in the project root folder on the Raspberry Pi.
2. Run the following command:  
`python pi_sender.py`
3. The Pi will begin capturing camera frames, extracting hand

landmarks, and streaming data to the Laptop over WebSocket.

---

## 5. Verifying the System is Running

---

- Laptop terminal should display model loaded confirmation and incoming Pi connection logs.
- Raspberry Pi terminal should show a successful WebSocket connection message.
- If using the web client, it can connect to the Laptop web server on port 3000 via a browser.

---

## 6. Common Issues & Resolutions

---

| Issue                           | Resolution Steps                                                                                                                                                                                                                                                                |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Connection Refused</b>       | <ul style="list-style-type: none"><li>• Verify the Laptop IP address is correctly set in pi_sender.py</li><li>• Confirm the Laptop firewall allows inbound traffic on ports 8765 and 3000</li><li>• Ensure laptop_rec.py was started before pi_sender.py</li></ul>              |
| <b>ModuleNotFoundError</b>      | <ul style="list-style-type: none"><li>• A required Python package is not installed</li><li>• Run: pip install &lt;missing-package-name&gt;</li><li>• Refer to Section 2 for the full package list per device</li></ul>                                                          |
| <b>Camera Not Opening (Pi)</b>  | <ul style="list-style-type: none"><li>• Check the camera ribbon cable connection to the Raspberry Pi</li><li>• Verify the camera device path (e.g. /dev/video0) is accessible</li><li>• Ensure the current user has permission to access the camera device</li></ul>            |
| <b>Model Not Found (Laptop)</b> | <ul style="list-style-type: none"><li>• Place the model file sign_language_model.pkl inside a models/ folder</li><li>• The expected path relative to project root is: models/sign_language_model.pkl</li><li>• Confirm the file name matches exactly (case-sensitive)</li></ul> |

---

## 7. Recommended Run Order

---

Always follow this sequence to ensure successful startup:

1. Start `laptop_rec.py` on the Windows Laptop (starts WebSocket + Web server)
2. Start `pi_sender.py` on the Raspberry Pi (connects to Laptop and begins streaming)

*For any additional configuration or troubleshooting, refer to the comments within each source file.*

## 9. Code

---

This is Laptop Cloude Code

"""

*Sign Language Receiver — Laptop*

*EasyOCR with full preprocessing pipeline:*

*flip → grayscale → 3× resize → CLAHE → sharpen → bilateral filter → adaptive threshold → EasyOCR*

*Pi sends a raw 1280×720 JPEG (no filters).*

*All image processing happens here on the laptop.*

*When a word is detected it is pushed back to the Pi as a 'show\_word' message*

*so the connected SH1106 OLED can display it.*

"""

```
from aiohttp import web
```

```
import asyncio
```

```
import websockets
```

```
import json
```

```
import numpy as np
```

```
import pickle
```

```
from collections import deque, Counter

import os

import sys

import aiohttp

import aiohttp_cors

import base64

from datetime import datetime

import traceback

import cv2


# =====
# OPTIONAL IMPORTS
# =====


EASYOCR_AVAILABLE = False

easyocr_reader = None

try:

    import easyocr

    EASYOCR_AVAILABLE = True

    print("[OK] easyocr imported")

except ImportError:

    print("[WARNING] easyocr not installed — run: pip install easyocr")

except Exception as e:

    print(f"[WARNING] easyocr import error: {e}")


WHISPER_AVAILABLE = False
```

```
whisper_model = None
```

```
try:
```

```
    import whisper
```

```
    WHISPER_AVAILABLE = True
```

```
    print("[OK] openai-whisper imported")
```

```
except ImportError:
```

```
    print("[WARNING] openai-whisper not installed — run: pip install openai-whisper")
```

```
except Exception as e:
```

```
    print(f"[WARNING] whisper import error: {e}")
```

```
try:
```

```
    import bluetooth
```

```
    BLUETOOTH_AVAILABLE = True
```

```
except ImportError:
```

```
    BLUETOOTH_AVAILABLE = False
```

```
GEMINI_AVAILABLE = False
```

```
try:
```

```
    from google import genai as google_genai
```

```
    GEMINI_AVAILABLE = True
```

```
    print("[OK] google-genai imported")
```

```
except ImportError:
```

```
    print("[WARNING] google-genai not installed — run: pip install google-genai")
```

```
TTS_AVAILABLE = False
```

```
try:
```

```
import pyttsx3

TTS_AVAILABLE = True

print("[OK] pyttsx3 imported")

except ImportError:

    print("[WARNING] pyttsx3 not installed — run: pip install pyttsx3")


if sys.platform == 'win32':

    import codecs

    sys.stdout = codecs.getwriter('utf-8')(sys.stdout.buffer, 'strict')

    sys.stderr = codecs.getwriter('utf-8')(sys.stderr.buffer, 'strict')


# =====
# CONFIGURATION
# =====


BASE_DIR = os.path.dirname(os.path.abspath(__file__))

MODEL_PATH = os.path.join(BASE_DIR, 'models', 'sign_language_model.pkl')


WEBSOCKET_PORT = 8765

WEB_SERVER_PORT = 3000


CONFIDENCE_THRESHOLD = 0.70

SMOOTHING_WINDOW = 7

MIN_CONSENSUS = 5

MAX_FRAME_SIZE = 10 * 1024 * 1024 # 10 MB

PRESETS_FILE = os.path.join(BASE_DIR, 'presets.json')
```



EASYOCR\_MIN\_CONFIDENCE = 0.3

GEMINI\_API\_KEY = "AlzaSyAp8a3uZqkDF-0xog3Flp4D0h2IgUC3kol"

GEMINI\_MODEL = "gemini-2.5-flash"

GEMINI\_PROMPT\_NO\_CONTEXT = ""\n

You are a sentence builder for a sign language recognition system.

The user will give you a list of words detected from sign language.

Your job is to form ONE single, complete, grammatically correct, natural-sounding sentence using ALL of those words. You have full creative freedom to add connecting words, articles, prepositions, verb tenses, or any other words needed to make the sentence sound fluent and natural.

Every word the user gives you MUST appear in the final sentence — do not drop any.

Reply with ONLY the sentence. No explanation, no punctuation outside the sentence, no quotes.\n

""\n

GEMINI\_PROMPT\_WITH\_CONTEXT = ""\n

You are a sentence builder for a sign language recognition system.

The user will give you:

1. A TOPIC / CONTEXT — this is the situation or subject being discussed.
2. A list of WORDS detected from sign language.

Your job is to form ONE single, complete, grammatically correct, natural-sounding sentence that fits within the given topic/context AND uses ALL of the provided words.

You have full creative freedom to add any connecting words needed to make it fluent.

Every provided word MUST appear in the final sentence — do not drop any.

Reply with ONLY the sentence. No explanation, no punctuation outside the sentence, no quotes.\

""""

TTS\_RATE = 160

```
KEYPAD_ACTION_LABELS = {  
    'scroll_down': 'Down Arrow',  
    'preset_1': 'Preset Slot 1',  
    'preset_2': 'Preset Slot 2',  
    'add_word': 'Add Word',  
    'preset_3': 'Preset Slot 3',  
    'standard_key_5': 'Standard key input',  
    'standard_key_6': 'Standard key input',  
    'send_to_ai': 'Gemini',  
    'standard_key_7': 'Standard key input',  
    'standard_key_8': 'Standard key input',  
    'standard_key_9': 'Standard key input',  
    'request_ocr': 'OCR',  
    'delete_last': 'Delete',  
    'scroll_up': 'Up Arrow',  
    'clear_words': 'Clear',  
    'toggle_audio': 'Record Toggle',  
}
```

# =====

*# GLOBAL STATE*

*# =====*

class SystemState:

def \_\_init\_\_(self):

self.model = None

self.scaler = None

self.label\_encoder = None

self.actions = []

self.prediction\_history = deque(maxlen=SMOOTHING\_WINDOW)

self.landmark\_buffer = deque(maxlen=5)

self.last\_prediction = None

self.prediction\_count = 0

self.words = []

self.context = ""

self.pi\_websocket = None

self.web\_clients = set()

self.pi\_connected = False

self.last\_word = ""

self.last\_confidence = 0.0

self.last\_frame\_base64 = None

self.last\_audio\_base64 = None

```
self.last_ocr_result = None

self.last_sentence = ""

self.last_dev_frame = None

self.dev_stream_enabled = False

self.presets = {
    1: "I want a coffee",
    2: "Call me later",
    3: "Help me",
}

self.last_keypad_event = None
```

```
self.error_log = deque(maxlen=50)

self.performance_stats = {
    'frames_received': 0,
    'predictions_made': 0,
    'avg_processing_time': 0,
    'ocr_calls': 0,
    'audio_calls': 0,
    'gemini_calls': 0,
}
```

```
state = SystemState()
```

```
def _load_presets():
    if not os.path.exists(PRESETS_FILE):
        return
```

```
try:
    with open(PRESETS_FILE, 'r', encoding='utf-8') as f:
        raw = json.load(f)
    for slot in (1, 2, 3):
        value = str(raw.get(str(slot), raw.get(slot, state.presets[slot]))).strip()
        state.presets[slot] = value
    print(f"[PRESET] Loaded presets from {PRESETS_FILE}")
except Exception as e:
    log_error("Loading presets", e, "Check presets.json format")
```

```
def _save_presets():
    try:
        with open(PRESETS_FILE, 'w', encoding='utf-8') as f:
            json.dump({str(k): v for k, v in state.presets.items()}, f, indent=2, ensure_ascii=False)
    except Exception as e:
        log_error("Saving presets", e, "Check file permissions for presets.json")
```

```
# =====
# ERROR HELPERS
# =====
```

```
def log_error(context, error, solution=""):
    entry = {
        'timestamp': datetime.now().isoformat(),
        'context': context,
        'error': str(error),
```

```
    'type':    type(error).__name__,
    'solution': solution,
    'traceback': traceback.format_exc(),
}
state.error_log.append(entry)
print(f"\n[ERROR] {context}")
print(f" Type:  {type(error).__name__}")
print(f" Message: {error}")
if solution:
    print(f" Fix:  {solution}")
return entry
```

```
async def broadcast_error(context, error, solution=""):
    entry = log_error(context, error, solution)
    await broadcast_to_web_clients({'type': 'error', 'error': entry})
```

```
# =====
# MODEL LOADING
# =====
```

```
def load_model():
    candidates = [
        MODEL_PATH,
        os.path.join('models', 'sign_language_model.pkl'),
        'sign_language_model.pkl',
    ]
```

```
for path in candidates:
    if not os.path.exists(path):
        continue
    try:
        with open(path, 'rb') as f:
            data = pickle.load(f)
            state.model      = data['model']
            state.scaler     = data['scaler']
            state.label_encoder = data['label_encoder']
            state.actions    = data['actions']
            print(f"[OK] Model loaded: {path}")
            print(f"[OK] Actions: {state.actions}")
            return True
    except Exception as e:
        log_error(f"Loading model: {path}", e, "Try retraining.")
log_error("Model not found", FileNotFoundError(), "Run: python train_model.py")
return False
```

```
# =====
```

```
# LANDMARK PROCESSING
```

```
# =====
```

```
def normalize_landmarks(arr):
    try:
        res = arr.copy() - arr[0]
        dist = np.max(np.linalg.norm(res, axis=1))
```

```
    if dist > 0:
        res /= dist
        mid = res[9]
        angle = np.arctan2(mid[1], mid[0])
        c, s = np.cos(-angle), np.sin(-angle)
        res[:, :2] = res[:, :2] @ np.array([[c, -s], [s, c]]).T
        return res.flatten()

except Exception as e:
    log_error("Landmark normalization", e)
    return None


def get_stabilized_landmarks(arr):
    n = normalize_landmarks(arr)
    if n is None:
        return None
    state.landmark_buffer.append(n)
    return np.mean(state.landmark_buffer, axis=0) if len(state.landmark_buffer) >= 3 else n


def preprocess_landmarks(raw):
    try:
        raw = np.array(raw, dtype=np.float32)
        hand = raw[0:63].reshape(21, 3)
        if np.all(hand == -1):
            hand2 = raw[64:127].reshape(21, 3)
            if np.all(hand2 == -1):
                return None
```



```
        hand = hand2

    return get_stabilized_landmarks(hand)

except Exception as e:

    log_error("Landmark preprocessing", e)

    return None


# =====
# PREDICTION
# =====


def predict_sign(landmarks):

    if state.model is None:

        return None, 0.0, []

    try:

        features = preprocess_landmarks(landmarks)

        if features is None:

            return None, 0.0, []

        features = state.scaler.transform(features.reshape(1, -1))

        probs = state.model.predict_proba(features)[0]

        idx = np.argmax(probs)

        word = state.label_encoder.classes_[idx]

        conf = float(probs[idx])

        top3 = [(state.label_encoder.classes_[i], float(probs[i]))

                for i in np.argsort(probs)[-3:][::-1]]

        state.prediction_count += 1

        state.performance_stats['predictions_made'] += 1
```

```
        return word, conf, top3

    except Exception as e:

        log_error("Prediction", e)

        return None, 0.0, []
```

```
def smooth_prediction(word, confidence):

    if confidence < CONFIDENCE_THRESHOLD:

        return None, False

    state.prediction_history.append(word)

    if len(state.prediction_history) < MIN_CONSENSUS:

        return None, False

    common, count = Counter(state.prediction_history).most_common(1)[0]

    if count >= MIN_CONSENSUS and common != state.last_prediction:

        state.last_prediction = common

        return common, True

    return None, False
```

```
# =====
# EASYOCR
# =====
```

```
def _preprocess_image_for_ocr(img_bgr):

    img_bgr = cv2.flip(img_bgr, 1)

    img_bgr = cv2.resize(img_bgr, None, fx=3, fy=3, interpolation=cv2.INTER_CUBIC)

    return img_bgr
```

```
def _run_easyocr(img_bgr):  
    if not EASYOCR_AVAILABLE or easyocr_reader is None:  
        raise RuntimeError("EasyOCR not initialised — run: pip install easyocr")  
  
    processed = _preprocess_image_for_ocr(img_bgr)  
    results = easyocr_reader.readtext(processed)  
  
    detections = []  
    text_parts = []  
  
    for bbox, text, prob in results:  
        print(f" [easyocr] {prob:.2f} {text!r}")  
  
        if prob >= EASYOCR_MIN_CONFIDENCE and text.strip():  
            text_parts.append(text.strip())  
  
            detections.append({  
                'text': text.strip(),  
                'confidence': round(float(prob), 3),  
                'bbox': [[int(p[0]), int(p[1])] for p in bbox],  
            })  
        else:  
            print(f" [skip] {prob:.2f} {text!r}")  
  
    return ' '.join(text_parts), detections  
  
def _encode_processed_preview(img_bgr):  
    try:  
        processed = _preprocess_image_for_ocr(img_bgr)  
  
        preview = cv2.resize(processed, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)  
  
        _, buf = cv2.imencode('.jpg', preview, [cv2.IMWRITE_JPEG_QUALITY, 80])  
  
        return base64.b64encode(buf.tobytes()).decode('utf-8')
```

except Exception:

    return None

# =====

# *GEMINI*

# =====

def \_call\_gemini(words: list, context: str) -> str:

    if not GEMINI\_AVAILABLE:

        raise RuntimeError("google-genai not installed — run: pip install google-genai")

    client = google\_genai.Client(api\_key=GEMINI\_API\_KEY)

    words\_str = ', '.join(words)

    if context.strip():

        prompt = (

            f'{GEMINI\_PROMPT\_WITH\_CONTEXT}\n\n'

            f'TOPIC / CONTEXT: {context.strip()}\n\n'

            f'WORDS: {words\_str}'

        )

    else:

        prompt = f'{GEMINI\_PROMPT\_NO\_CONTEXT}\n\nWORDS: {words\_str}'

    print(f'[GEMINI] Sending → words={words\_str!r} context={context.strip()!r}')

    response = client.models.generate\_content(model=GEMINI\_MODEL, contents=prompt)

    sentence = response.text.strip().strip('"').strip('\"')

    print(f'[GEMINI] Response: {sentence!r}')

    return sentence

```
# =====
```

```
# TTS
```

```
# =====
```

```
def _speak(text: str):
```

```
    if not TTS_AVAILABLE:
```

```
        raise RuntimeError("pyttsx3 not installed — run: pip install pyttsx3")
```

```
    engine = pyttsx3.init()
```

```
    engine.setProperty('rate', TTS_RATE)
```

```
    engine.setProperty('volume', 1.0)
```

```
    engine.say(text)
```

```
    engine.runAndWait()
```

```
    engine.stop()
```

```
    print(f"[TTS] Spoke: {text!r}")
```

```
# =====
```

```
# BROADCAST — goes to ALL web clients AND the Pi OLED
```

```
# =====
```

```
async def broadcast_to_web_clients(message):
```

```
    """
```

```
    Send to every browser client AND forward to Pi so the OLED stays in sync.
```

```
    'show_word' is excluded here because send_word_to_web() handles it directly.
```

```
    """
```

```
# — Browser clients —————
```

```
    dead = set()
```

```
for client in state.web_clients:
```

```
    try:
```

```
        await client.send_str(json.dumps(message))
```

```
    except Exception:
```

```
        dead.add(client)
```

```
state.web_clients -= dead
```

```
# — Pi / OLED — forward everything except show_word —————
```

```
if state.pi_websocket and message.get('type') != 'show_word':
```

```
    try:
```

```
        await state.pi_websocket.send(json.dumps(message))
```

```
    except Exception:
```

```
        pass
```

```
async def send_word_to_web(word, confidence):
```

```
    """Broadcast detected word to browser clients AND push show_word to Pi OLED."""
```

```
    await broadcast_to_web_clients({
```

```
        'type':    'word_detected',
```

```
        'word':    word,
```

```
        'confidence': confidence,
```

```
        'timestamp': datetime.now().isoformat(),
```

```
    })
```

```
# Direct show_word push to Pi for the OLED prediction strip
```

```
if state.pi_websocket:
```

```
    try:
```

```
        await state.pi_websocket.send(json.dumps({
```

```
        'type':    'show_word',
        'word':    word,
        'confidence': confidence,
    )))
except Exception:
    pass
```

```
async def send_status_to_web():
    await broadcast_to_web_clients({
        'type':        'status',
        'pi_connected': state.pi_connected,
        'words_count':  len(state.words),
        'last_word':    state.last_word,
        'last_confidence': state.last_confidence,
        'stats':        state.performance_stats,
    })
```

```
async def send_presets_to_pi():
    if not state.pi_websocket:
        return
    try:
        await state.pi_websocket.send(json.dumps({
            'type': 'presets_updated',
            'presets': {str(slot): text for slot, text in state.presets.items()},
        }))
    except Exception as e:
```

```
log_error("Send presets to Pi", e)
```

```
# =====
```

```
# PI WEBSOCKET HANDLER
```

```
# =====
```

```
async def handle_pi_connection(websocket):
```

```
    state.pi_websocket = websocket
```

```
    state.pi_connected = True
```

```
    print("[PI] Connected!")
```

```
    await send_presets_to_pi()
```

```
    await send_status_to_web()
```

```
    try:
```

```
        async for message in websocket:
```

```
            try:
```

```
                state.performance_stats['frames_received'] += 1
```

```
                data = json.loads(message)
```

```
                msg_type = data.get('type') if isinstance(data, dict) else None
```

```
                if msg_type == 'ping':
```

```
                    continue
```

```
                elif msg_type == 'frame':
```

```
                    await handle_frame_from_pi(data)
```

```
                    continue
```

```
                elif msg_type == 'audio':
```



```
    await handle_audio_from_pi(data)

    continue

elif msg_type == 'audio_status':

    await broadcast_to_web_clients(data)

    continue

elif msg_type == 'dev_frame':

    frame_b64 = data.get('frame')

    if frame_b64:

        state.last_dev_frame = frame_b64

        await broadcast_to_web_clients({

            'type': 'dev_frame',

            'image': frame_b64,

            'timestamp': data.get('timestamp'),

        })

    continue

elif msg_type == 'dev_stream_status':

    state.dev_stream_enabled = bool(data.get('enabled', False))

    await broadcast_to_web_clients({

        'type': 'dev_stream_status',

        'enabled': state.dev_stream_enabled,

    })

    continue

elif msg_type == 'keypad_press':

    state.last_keypad_event = {

        'key': data.get('key', ""),

        'action': data.get('action', ""),
```

```
        'timestamp': datetime.now().isoformat(),
    }
    await broadcast_to_web_clients({
        'type': 'keypad_press',
        'key': data.get('key', ""),
        'action': data.get('action', ""),
        'action_label': KEYPAD_ACTION_LABELS.get(data.get('action', ""),
data.get('action', "")),
    })
    continue
```

```
if isinstance(data, dict) and data.get('command'):
    await handle_web_command(data, None, source='pi')
    continue
```

```
# — Landmark / sign data —————
```

```
landmarks = data.get('landmarks') if isinstance(data, dict) else data
```

```
if landmarks is None:
```

```
    continue
```

```
landmarks = np.array(landmarks, dtype=np.float32)
```

```
if landmarks.shape[0] != 128:
```

```
    continue
```

```
word, conf, _ = predict_sign(landmarks)
```

```
if word is None:
```

```
    continue
```

```
final, show = smooth_prediction(word, conf)
```

```
if show:
```

```
        state.last_word      = final

        state.last_confidence = conf

        print(f"[DETECTED] {final.upper()} ({conf:.1%})")

        await send_word_to_web(final, conf)

    except json.JSONDecodeError as e:

        await broadcast_error("JSON parse from Pi", e, "Pi must send valid JSON")

    except Exception as e:

        await broadcast_error("Processing Pi message", e)

except websockets.exceptions.ConnectionClosed as e:

    print(f"[PI] Connection closed: {getattr(e, 'reason', 'unknown')}")

except Exception as e:

    await broadcast_error("Pi connection handler", e)

finally:

    state.pi_connected = False

    state.pi_websocket = None

    print("[PI] Disconnected")

    await send_status_to_web()

# =====

# OCR FRAME HANDLER

# =====

async def handle_frame_from_pi(data):

    try:
```

```
state.performance_stats['ocr_calls'] += 1
```

```
if not EASYOCR_AVAILABLE or easyocr_reader is None:
```

```
    raise RuntimeError("EasyOCR not initialised — run: pip install easyocr")
```

```
frame_b64 = data.get('frame', "")
```

```
if not frame_b64:
```

```
    raise ValueError("Pi sent an empty 'frame' field")
```

```
clean = frame_b64.strip().replace('\n', "").replace('\r', "").replace(' ', "")
```

```
if not clean:
```

```
    raise ValueError("base64 string is empty")
```

```
try:
```

```
    img_bytes = base64.b64decode(clean + '==')
```

```
except Exception as e:
```

```
    raise ValueError(f"base64 decode failed: {e}")
```

```
if len(img_bytes) < 500:
```

```
    raise ValueError(f"Decoded frame only {len(img_bytes)} bytes")
```

```
print(f"[OCR] Frame received: {len(img_bytes) // 1024} KB")
```

```
state.last_frame_base64 = frame_b64
```

```
arr = np.frombuffer(img_bytes, dtype=np.uint8)
```

```
img_bgr = cv2.imdecode(arr, cv2.IMREAD_COLOR)
```

```
if img_bgr is None:
    raise ValueError("cv2.imdecode failed — not a valid JPEG")

h, w = img_bgr.shape[:2]
print(f"[OCR] Image decoded: {w}×{h} → EasyOCR ...")

loop = asyncio.get_event_loop()
try:
    text, detections = await asyncio.wait_for(
        loop.run_in_executor(None, _run_easyocr, img_bgr),
        timeout=60.0
    )
except asyncio.TimeoutError:
    raise RuntimeError("OCR timed out after 60 s")

print(f"[OCR] Extracted ({len(text)} chars): {text[:200]!r}")

processed_preview_b64 = await loop.run_in_executor(
    None, _encode_processed_preview, img_bgr
)

state.last_ocr_result = {
    'text': text,
    'detections': detections,
    'timestamp': datetime.now().isoformat(),
}
```

if text:

```
    await broadcast_to_web_clients({
        'type': 'ocr_result',
        'text': text,
        'detections': detections,
        'image_preview': frame_b64,
        'processed_preview': processed_preview_b64,
        'needs_confirmation': True,
    })
```

else:

```
    await broadcast_to_web_clients({
        'type': 'error',
        'error': {
            'context': 'OCR',
            'error': 'No text detected in image',
            'solution': 'Ensure text is clear, well-lit and camera is steady.',
        },
    })
```

except ValueError as e:

```
    await broadcast_error("OCR - bad image data", e, "Check Pi is sending a valid base64 JPEG")
```

except RuntimeError as e:

```
    await broadcast_error("OCR - engine error", e, "run: pip install easyocr")
```

except Exception as e:

```
    await broadcast_error("OCR processing", e, "Check easyocr installation")
```

```
# =====  
  
# AUDIO HANDLER  
  
# =====  
  
def _transcribe_with_whisper(audio_bytes, rate, channels):  
    audio_np = np.frombuffer(audio_bytes, dtype=np.int16).astype(np.float32) / 32768.0  
  
    if channels == 2:  
        audio_np = audio_np.reshape(-1, 2).mean(axis=1)  
  
    if rate != 16000:  
        target_len = int(len(audio_np) * 16000 / rate)  
  
        audio_np = np.interp(  
            np.linspace(0, len(audio_np) - 1, target_len),  
            np.arange(len(audio_np)),  
            audio_np  
        ).astype(np.float32)  
  
    print(f"[AUDIO] Running Whisper on {len(audio_np)/16000:.1f}s ...")  
    result = whisper_model.transcribe(audio_np, fp16=False, language='en')  
    return result.get('text', "").strip()  
  
async def handle_audio_from_pi(data):  
    try:  
        state.performance_stats['audio_calls'] += 1  
  
        if not WHISPER_AVAILABLE or whisper_model is None:  
            raise RuntimeError("Whisper not initialised — run: pip install openai-whisper")
```

```
audio_b64 = data.get('audio', '')
duration = data.get('duration', 0)
rate     = data.get('rate', 16000)
channels = data.get('channels', 1)

if not audio_b64:
    raise ValueError("No audio data received from Pi")

state.last_audio_base64 = audio_b64
print(f"[AUDIO] Received {duration:.1f}s at {rate} Hz ({channels}ch)")

audio_bytes = base64.b64decode(audio_b64)

await broadcast_to_web_clients({
    'type': 'audio_status',
    'status': 'transcribing',
    'duration': duration,
})

loop = asyncio.get_event_loop()
try:
    text = await asyncio.wait_for(
        loop.run_in_executor(
            None, _transcribe_with_whisper, audio_bytes, rate, channels
        ),
```



```
        timeout=120.0

    )

except asyncio.TimeoutError:

    raise RuntimeError("Whisper transcription timed out after 120 s")

print(f"[AUDIO] Transcribed: {text!r}")

if text:

    await broadcast_to_web_clients({

        'type':      'audio_result',

        'text':      text,

        'duration':   duration,

        'needs_confirmation': True,

    })

else:

    await broadcast_to_web_clients({

        'type': 'error',

        'error': {

            'context': 'Audio',

            'error':   'Whisper returned empty transcription',

            'solution': 'Speak clearly — check microphone and try again.',

        },

    })

except ValueError as e:

    await broadcast_error("Audio - bad data", e, "Check Pi is sending valid audio")
```

```
except RuntimeError as e:
```

```
    await broadcast_error("Audio - engine error", e, "run: pip install openai-whisper")
```

```
except Exception as e:
```

```
    await broadcast_error("Audio processing", e)
```

```
# =====
```

```
# SEND COMMAND TO PI
```

```
# =====
```

```
async def send_command_to_pi(command, params=None):
```

```
    if not state.pi_websocket:
```

```
        return {'success': False, 'error': 'Pi not connected'}
```

```
    try:
```

```
        await state.pi_websocket.send(json.dumps({
```

```
            'type':    'command',
```

```
            'command': command,
```

```
            'params':  params or {},
```

```
            'timestamp': datetime.now().isoformat(),
```

```
        )))
```

```
        print(f"[CMD] -> Pi: {command}")
```

```
        return {'success': True}
```

```
    except Exception as e:
```

```
        log_error("Send command to Pi", e)
```

```
        return {'success': False, 'error': str(e)}
```

```
# =====
```

# *BROWSER AUDIO HANDLER*

# =====

async def handle\_browser\_audio(data, ws):

try:

state.performance\_stats['audio\_calls'] += 1

if not WHISPER\_AVAILABLE or whisper\_model is None:

raise RuntimeError("Whisper not loaded — run: pip install openai-whisper")

audio\_b64 = data.get('audio', '')

duration = data.get('duration', 0)

if not audio\_b64:

raise ValueError("No audio data in browser\_audio command")

await broadcast\_to\_web\_clients({

'type': 'audio\_status',

'status': 'transcribing',

'duration': duration,

})

audio\_bytes = base64.b64decode(audio\_b64)

audio\_np = np.frombuffer(audio\_bytes, dtype=np.float32).copy()

audio\_np = np.clip(audio\_np, -1.0, 1.0)

```
if len(audio_np) < 1600:

    raise ValueError("Audio too short — hold the button longer")

loop = asyncio.get_event_loop()

try:

    text = await asyncio.wait_for(

        loop.run_in_executor(None, _transcribe_numpy, audio_np),

        timeout=120.0

    )

except asyncio.TimeoutError:

    raise RuntimeError("Whisper timed out after 120 s")

print(f"[BROWSER AUDIO] Transcribed: {text!r}")

if text:

    await broadcast_to_web_clients({

        'type':      'audio_result',

        'text':      text,

        'duration':   duration,

        'source':     'browser',

        'needs_confirmation': True,

    })

else:

    await broadcast_to_web_clients({

        'type': 'error',

        'error': {
```

```
        'context': 'Browser Audio',
        'error': 'Whisper returned empty result',
        'solution': 'Speak clearly and close to the microphone.',
    },
))
```

```
except ValueError as e:
```

```
    await broadcast_to_web_clients({'type': 'error', 'error': {
        'context': 'Browser Audio', 'error': str(e), 'solution': ''}})
```

```
except RuntimeError as e:
```

```
    await broadcast_to_web_clients({'type': 'error', 'error': {
        'context': 'Browser Audio', 'error': str(e),
        'solution': 'pip install openai-whisper'}})
```

```
except Exception as e:
```

```
    log_error("Browser audio", e)
    await broadcast_to_web_clients({'type': 'error', 'error': {
        'context': 'Browser Audio', 'error': str(e), 'solution': ''}})
```

```
def _transcribe_numpy(audio_np):
```

```
    result = whisper_model.transcribe(audio_np, fp16=False, language='en')
    return result.get('text', "").strip()
```

```
# =====
# WEB SERVER
# =====
```

```
async def websocket_handler(request):

    ws = web.WebSocketResponse()

    await ws.prepare(request)

    state.web_clients.add(ws)

    print(f"[WEB] Client connected ({len(state.web_clients)} total)")


    await ws.send_str(json.dumps({

        'type':      'init',

        'pi_connected': state.pi_connected,

        'words':      state.words,

        'context':     state.context,

        'actions':     state.actions,

        'presets':     {str(slot): text for slot, text in state.presets.items()},

        'keypad_map':  KEYPAD_ACTION_LABELS,

        'last_keypad_event': state.last_keypad_event,

        'dev_stream_enabled': state.dev_stream_enabled,

        'stats':       state.performance_stats,

        'features': {

            'ocr':      EASYOCR_AVAILABLE,

            'audio':     WHISPER_AVAILABLE,

            'bluetooth': BLUETOOTH_AVAILABLE,

            'gemini':    GEMINI_AVAILABLE,

            'tts':       TTS_AVAILABLE,

        },

    })))
```

```
try:
    async for msg in ws:
        if msg.type == aiohttp.WSMsgType.TEXT:
            try:
                await handle_web_command(json.loads(msg.data), ws)
            except Exception as e:
                await broadcast_error("Web command", e)
        elif msg.type == aiohttp.WSMsgType.ERROR:
            print(f"[WEB] Error: {ws.exception()}")
    finally:
        state.web_clients.discard(ws)
        print(f"[WEB] Client disconnected ({len(state.web_clients)} remaining)")
    return ws
```

```
async def _send_ws_if_present(ws, payload):
```

```
    if ws is None:
        return
    await ws.send_str(json.dumps(payload))
```

```
async def handle_web_command(data, ws, source='web'):
```

```
    cmd = data.get('command')
    print(f"[CMD:{source.upper()}] {cmd}")
```

```
# — Preset Trigger (from Pi keypad) —————
```

```
if cmd == 'trigger_preset':
```

```
    try:
```

```
        slot = int(data.get('slot', 0) or 0)
except (TypeError, ValueError):
    slot = 0

sentence = str(data.get('sentence', "")).strip()

if not sentence and slot in state.presets:
    sentence = state.presets[slot]

if not sentence:
    await broadcast_to_web_clients({'type': 'error', 'error': {
        'context': 'Preset',
        'error': f'Preset slot {slot} is empty',
        'solution': 'Set preset text in the UI and save it',
    }})

    return

state.words.append(sentence)

await broadcast_to_web_clients({
    'type': 'preset_triggered',
    'slot': slot,
    'sentence': sentence,
    'source': source,
})

await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})

return
```

# — *Standard Key Input (5/6/7/8/9)* —————

```
if cmd == 'standard_key_input':
    key = str(data.get('key', "")).strip()
```



```
if key:
    state.words.append(key)
    await broadcast_to_web_clients({
        'type': 'standard_key_input',
        'key': key,
        'source': source,
    })
    await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})
return
```

# — *Preset Update (from Web UI)* —————

```
if cmd == 'update_preset':
    try:
        slot = int(data.get('slot', 0) or 0)
    except (TypeError, ValueError):
        slot = 0
    text = str(data.get('text', "")).strip()
    if slot not in (1, 2, 3):
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context': 'Preset',
            'error': f'Invalid preset slot: {slot}',
            'solution': 'Use only preset slots 1, 2 or 3',
        }})
    return
    state.presets[slot] = text
    _save_presets()
```

```
await broadcast_to_web_clients({
    'type': 'presets_updated',
    'presets': {str(k): v for k, v in state.presets.items()},
    'updated_slot': slot,
})
await send_presets_to_pi()
return
```

```
# — Add Word —————  
  
# Accepts an explicit 'word' field (from Pi keypad) OR falls  
# back to state.last_word (from web UI Add Word button).  
if cmd == 'add_word':  
    word = data.get('word', "").strip()  
  
    if not word:  
        word = state.last_word.strip()  
  
    if word:  
        state.words.append(word)  
        print(f"[ADD WORD] '{word}' → words now: {state.words}")  
        await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})  
  
    else:  
        # Nothing to add — tell the client  
        await broadcast_to_web_clients({'type': 'error', 'error': {  
            'context': 'Add Word',  
            'error': 'No word detected yet — sign a word first',  
            'solution': 'Make a sign and wait for the prediction to appear',  
        }})
```

```
return
```

```
# — Delete Last —————
```

```
elif cmd == 'delete_last':
```

```
    if state.words:
```

```
        removed = state.words.pop()
```

```
        print(f"[DELETE] Removed '{removed}' → words now: {state.words}")
```

```
        await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})
```

```
# — Clear All —————
```

```
elif cmd == 'clear_words':
```

```
    state.words = []
```

```
    print("[CLEAR] All words cleared")
```

```
    await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})
```

```
# — Send to Gemini —————
```

```
elif cmd == 'send_to_ai':
```

```
    if not state.words:
```

```
        await broadcast_to_web_clients({'type': 'error', 'error': {
```

```
            'context': 'Gemini',
```

```
            'error': 'No words to send — add some words first',
```

```
            'solution': 'Sign some words and add them before sending to AI',
```

```
        }})
```

```
    return
```

```
if not GEMINI_AVAILABLE:
```

```
await broadcast_to_web_clients({'type': 'error', 'error': {
    'context': 'Gemini',
    'error': 'google-generativeai not installed',
    'solution': 'pip install google-generativeai',
}})

return

# Notify ALL clients (browser + Pi OLED) that Gemini is generating

await broadcast_to_web_clients({'type': 'gemini_status', 'status': 'generating'})

state.performance_stats['gemini_calls'] += 1
words_snapshot = list(state.words)
context_snapshot = state.context

loop = asyncio.get_event_loop()
try:
    sentence = await asyncio.wait_for(
        loop.run_in_executor(
            None, _call_gemini, words_snapshot, context_snapshot
        ),
        timeout=30.0
    )
except asyncio.TimeoutError:
    await broadcast_to_web_clients({'type': 'error', 'error': {
        'context': 'Gemini',
        'error': 'Request timed out after 30 s',
```

```
        'solution': 'Check your internet connection and API key',
    })

    await broadcast_to_web_clients({'type': 'gemini_status', 'status': 'idle'})

    return

except Exception as e:

    log_error("Gemini call", e, "Check GEMINI_API_KEY in laptop_receiver.py")

    await broadcast_to_web_clients({'type': 'error', 'error': {

        'context': 'Gemini',

        'error': str(e),

        'solution': 'Check GEMINI_API_KEY in laptop_receiver.py',

    }})

    await broadcast_to_web_clients({'type': 'gemini_status', 'status': 'idle'})

    return


state.last_sentence = sentence


# Broadcast sentence to ALL clients (browser + Pi OLED)

await broadcast_to_web_clients({

    'type': 'gemini_sentence',

    'sentence': sentence,

    'words': words_snapshot,

    'context': context_snapshot,

})


if TTS_AVAILABLE:

    loop.run_in_executor(None, _speak, sentence)
```

```
else:
```

```
    print("[TTS] Skipped — pyttsx3 not installed")
```

```
# — Speak Sentence —————
```

```
elif cmd == 'speak_sentence':
```

```
    text = data.get('sentence', state.last_sentence)
```

```
    if text and TTS_AVAILABLE:
```

```
        loop = asyncio.get_event_loop()
```

```
        loop.run_in_executor(None, _speak, text)
```

```
    elif not TTS_AVAILABLE:
```

```
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context': 'TTS', 'error': 'pyttsx3 not installed',
            'solution': 'pip install pyttsx3'}})
```

```
# — Context —————
```

```
elif cmd == 'remove_context':
```

```
    state.context = ""
```

```
    await broadcast_to_web_clients({'type': 'context_updated', 'context': state.context})
```

```
elif cmd == 'add_context':
```

```
    state.context = (state.context + ' ' + data.get('text', "")).strip()
```

```
    await broadcast_to_web_clients({'type': 'context_updated', 'context': state.context})
```

```
# — OCR —————
```

```
elif cmd == 'request_ocr':
```

```
    result = await send_command_to_pi('capture_frame')
```

```
await _send_ws_if_present(ws, result)
```

```
# — Audio —————
```

```
elif cmd == 'browser_audio':
```

```
    await handle_browser_audio(data, ws)
```

```
elif cmd == 'request_audio':
```

```
    duration = data.get('duration', 10)
```

```
    result = await send_command_to_pi('record_audio', {'duration': duration})
```

```
    await _send_ws_if_present(ws, result)
```

```
elif cmd == 'start_audio':
```

```
    result = await send_command_to_pi('start_audio')
```

```
    await _send_ws_if_present(ws, result)
```

```
elif cmd == 'stop_audio':
```

```
    result = await send_command_to_pi('stop_audio')
```

```
    await _send_ws_if_present(ws, result)
```

```
elif cmd == 'toggle_dev_stream':
```

```
    enabled = bool(data.get('enabled', False))
```

```
    result = await send_command_to_pi('toggle_dev_stream', {'enabled': enabled})
```

```
    await _send_ws_if_present(ws, result)
```

```
    if result.get('success'):
```

```
        state.dev_stream_enabled = enabled
```

```
        await broadcast_to_web_clients({'type': 'dev_stream_status', 'enabled': enabled})
```

else:

state.dev\_stream\_enabled = False

await broadcast\_to\_web\_clients({'type': 'dev\_stream\_status', 'enabled': False})

await broadcast\_to\_web\_clients({'type': 'error', 'error': {

    'context': 'Dev Stream',

    'error': result.get('error', 'Pi not connected'),

    'solution': 'Start pi\_sender.py and ensure Pi websocket is connected',

}})

# — *Bluetooth* —————

elif cmd == 'scan\_bluetooth':

devices = []

if BLUETOOTH\_AVAILABLE:

try:

raw = bluetooth.discover\_devices(duration=5, lookup\_names=True)

devices = [{'address': a, 'name': n} for a, n in raw]

except Exception as e:

log\_error("Bluetooth scan", e)

await \_send\_ws\_if\_present(ws, {'type': 'bluetooth\_devices', 'devices': devices})

# — *Debug* —————

elif cmd == 'get\_debug\_data':

await \_send\_ws\_if\_present(ws, {

    'type': 'debug\_data',

    'last\_frame': state.last\_frame\_base64,

    'last\_dev\_frame': state.last\_dev\_frame,



```
'last_audio': state.last_audio_base64,  
'last_ocr': state.last_ocr_result,  
'dev_stream_enabled': state.dev_stream_enabled,  
'error_log': list(state.error_log)[-10:],  
'stats': state.performance_stats,  
})
```

```
async def serve_index(request):  
    path = os.path.join(BASE_DIR, 'web', 'index.html')  
    return web.FileResponse(path) if os.path.exists(path) else \  
        web.Response(text="Web interface not found", status=404)
```

```
# =====  
# MAIN  
# =====
```

```
async def main():  
    print("\n" + "=" * 60)  
    print("SIGN LANGUAGE RECOGNITION SYSTEM")  
    print("=" * 60)  
  
    if not load_model():  
        print("\n[FATAL] No model — run: python train_model.py")  
        return  
  
    _load_presets()
```

```
print(f"\n[CONFIG]")

print(f" WebSocket port : {WEBSOCKET_PORT}")

print(f" Web server port: {WEB_SERVER_PORT}")

print(f" EasyOCR      : {'available' if EASYOCR_AVAILABLE else 'not available'}")

print(f" Whisper      : {'available' if WHISPER_AVAILABLE else 'not available'}")

print(f" Gemini        : {'available' if GEMINI_AVAILABLE else 'not available'}")

print(f" TTS (pyttsx3) : {'available' if TTS_AVAILABLE else 'not available'}")
```

```
global easyocr_reader
```

```
if EASYOCR_AVAILABLE:
```

```
    try:
```

```
        print("[OCR] Loading EasyOCR model ...")
```

```
        easyocr_reader = easyocr.Reader(['en'], gpu=False)
```

```
        dummy = np.zeros((64, 64, 3), dtype=np.uint8)
```

```
        easyocr_reader.readtext(dummy)
```

```
        print("[OCR] EasyOCR ready ✓")
```

```
    except Exception as e:
```

```
        print(f"[OCR] WARNING: {e}")
```

```
        easyocr_reader = None
```

```
global whisper_model
```

```
if WHISPER_AVAILABLE:
```

```
    try:
```

```
        print("[AUDIO] Loading Whisper 'base' model ...")
```

```
        whisper_model = whisper.load_model('base')
```

```
        print("[AUDIO] Whisper ready ✓")

    except Exception as e:

        print(f"[AUDIO] WARNING: {e}")

        whisper_model = None


pi_server = await websockets.serve(

    handle_pi_connection, "0.0.0.0", WEBSOCKET_PORT,

    ping_interval=20, ping_timeout=20,

    max_size=MAX_FRAME_SIZE,

)

print(f"[OK] Pi WebSocket on port {WEBSOCKET_PORT}")


app = web.Application()

cors = aiohttp_cors.setup(app, defaults={

    "**": aiohttp_cors.ResourceOptions(

        allow_credentials=True, expose_headers="**", allow_headers="**"

    )

})

app.router.add_get('/', serve_index)

app.router.add_get('/ws', websocket_handler)

app.router.add_static('/static', os.path.join(BASE_DIR, 'web', 'static'), name='static')

for route in list(app.router.routes()):

    cors.add(route)


runner = web.AppRunner(app)

await runner.setup()

await web.TCPSite(runner, '0.0.0.0', WEB_SERVER_PORT).start()
```

```
print(f"[OK] Web server on http://localhost:{WEB_SERVER_PORT}")

print("=" * 60)

print(f" EasyOCR : {'✓ loaded' if easyocr_reader else 'X not loaded'}")

print(f" Whisper : {'✓ loaded' if whisper_model else 'X not loaded'}")

print(f" Gemini : {'✓ ready' if GEMINI_AVAILABLE else 'X not installed'}")

print(f" TTS : {'✓ ready' if TTS_AVAILABLE else 'X not installed'}")

print("Open browser and start Pi sender")

print("=" * 60 + "\n")

await asyncio.Future()

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("\n[SHUTDOWN] Stopped")
    except Exception as e:
        log_error("Main", e)
```

## PI 4 Code

```
import cv2

import mediapipe as mp

import websocket

import json
```

```
import time
```

```
import socket
```

```
import numpy as np
```

```
import base64
```

```
import threading
```

```
try:
```

```
    import sounddevice as sd
```

```
    AUDIO_AVAILABLE = True
```

```
except ImportError:
```

```
    AUDIO_AVAILABLE = False
```

```
    print("[WARNING] sounddevice not installed — run: pip install sounddevice")
```

```
try:
```

```
    from luma.core.interface.serial import i2c
```

```
    from luma.oled.device import sh1106
```

```
    from luma.core.render import canvas
```

```
    from PIL import ImageFont
```

```
    OLED_AVAILABLE = True
```

```
except ImportError:
```

```
    OLED_AVAILABLE = False
```

```
    print("[WARNING] luma.oled not installed — run: pip install luma.oled")
```

```
try:
```

```
    import RPi.GPIO as GPIO
```

```
    GPIO_AVAILABLE = True
```

except ImportError:

GPIO\_AVAILABLE = False

print("[WARNING] RPi.GPIO not available")

# =====

# CONFIGURATION

# =====

PC\_IP = "DESKTOP-EG03B2I.local"

WS\_URL = f"ws://{PC\_IP}:8765"

TARGET\_FPS = 30

SEND\_INTERVAL = 1.0 / TARGET\_FPS

HEARTBEAT\_INTERVAL = 2.0

MIN\_DETECTION\_CONFIDENCE = 0.7

MIN\_TRACKING\_CONFIDENCE = 0.6

MAX\_NUM\_HANDS = 2

SHOW\_LANDMARKS = True

CAPTURE\_WIDTH = 1280

CAPTURE\_HEIGHT = 720

CAPTURE\_JPEG\_QUALITY = 85

AUDIO\_SAMPLE\_RATE = 16000

AUDIO\_CHANNELS = 1

AUDIO\_MAX\_SECONDS = 120

# =====

# KEYPAD SETUP

# =====

ROWS = [5, 6, 13, 19]

COLS = [12, 16, 20, 21]

KEYS = [

    ['1', '2', '3', 'A'],

    ['4', '5', '6', 'B'],

    ['7', '8', '9', 'C'],

    ['\*', '0', '#', 'D'],

]

# Key → action mapping

# 2,3 = Preset 1,2 (row 1)

# 4,5,6 = Preset 3,4,5 (row 2)

# 7,8,9 = Preset 6,7,8 (row 3)

# 0 = scroll up, 1 = scroll down

KEY\_ACTIONS = {

    'A': 'add\_word',

    'B': 'send\_to\_ai',

    'C': 'request\_ocr',

    'D': 'toggle\_audio',

    '\*': 'delete\_last',

```
'#': 'clear_words',
'0': 'scroll_up',
'1': 'scroll_down',
'2': 'preset_1',
'3': 'preset_2',
'4': 'preset_3',
'5': 'preset_4',
'6': 'preset_5',
'7': 'preset_6',
'8': 'preset_7',
'9': 'preset_8',
}

# =====
# MEDIAPIPE
# =====

mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils

hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=MAX_NUM_HANDS,
    min_detection_confidence=MIN_DETECTION_CONFIDENCE,
    min_tracking_confidence=MIN_TRACKING_CONFIDENCE,
)
```



```
landmark_spec = mp_drawing.DrawingSpec(color=(0, 0, 255), thickness=2, circle_radius=3)
```

```
connection_spec = mp_drawing.DrawingSpec(color=(0, 255, 0), thickness=2)
```

```
# =====
```

```
# GLOBALS
```

```
# =====
```

```
last_send_time = 0
```

```
last_heartbeat_time = 0
```

```
frame_count = 0
```

```
fps_val = 0
```

```
last_fps_time = time.time()
```

```
latest_frame_lock = threading.Lock()
```

```
latest_frame = None
```

```
audio_lock = threading.Lock()
```

```
audio_recording = False
```

```
audio_stop_event = threading.Event()
```

```
audio_recorded_chunks = []
```

```
# — OLED state —————

---


```

```
display_lock = threading.Lock()
```

```
oled_prediction = ""
```

```
oled_words = []
```

```
oled_context = ""
```

```
oled_status      = "READY"
```

```
oled_flash_text  = ""
```

```
oled_flash_until = 0.0
```

```
oled_scroll_offset = 0
```

```
oled_is_recording = False
```

```
oled_event = threading.Event()
```

```
ws_ref_holder = [None]
```

```
# — Preset cache (populated from laptop on connect) —————
```

```
preset_cache = {i: "" for i in range(1, 9)} # 1-8
```

```
preset_lock = threading.Lock()
```

```
# =====
```

```
# FONTS
```

```
# =====
```

```
font_large = None
```

```
font_medium = None
```

```
font_small = None
```

```
def _load_fonts():
```

```
    global font_large, font_medium, font_small
```

```
    bold = "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf"
```

```
    normal = "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf"
```

```
try:
    font_large = ImageFont.truetype(bold, 15)
    font_medium = ImageFont.truetype(normal, 11)
    font_small = ImageFont.truetype(normal, 10)
except Exception:
    font_large = font_medium = font_small = None
```

```
# =====
# OLED LAYOUT
# =====
```

```
DISPLAY_W = 128
DISPLAY_H = 64
STATUS_H = 12
DIV_Y = STATUS_H
PRED_H = 16
PRED_Y = DISPLAY_H - PRED_H
SENT_Y = DIV_Y + 2
SENT_H = PRED_Y - SENT_Y - 2
LINE_H = 12
```

```
def _tw(draw, text, font):
    try:
        b = draw.textbbox((0, 0), text, font=font)
        return b[2] - b[0]
```

except Exception:

    return len(text) \* 6

def \_wrap(draw, text, font, max\_w):

    if not text:

        return [""]

    words, lines, cur = text.split(), [], ""

    for w in words:

        test = (cur + " " + w).strip()

        if \_tw(draw, test, font) <= max\_w:

            cur = test

        else:

            if cur:

                lines.append(cur)

            cur = w

    if cur:

        lines.append(cur)

    return lines or [""]

def \_draw\_normal\_ui(draw, status, is\_rec, words, scroll, prediction):

    f\_s = font\_small or ImageFont.load\_default()

    f\_m = font\_medium or ImageFont.load\_default()

    f\_l = font\_large or ImageFont.load\_default()

# Status bar

```
draw.rectangle([0, 0, DISPLAY_W - 1, STATUS_H - 1], fill="black")
```

```
label = ("* " if is_rec else "") + status
```

```
while _tw(draw, label, f_s) > DISPLAY_W - 4 and len(label) > 1:
```

```
    label = label[:-1]
```

```
draw.text((2, 1), label, font=f_s, fill="white")
```

```
draw.line([0, DIV_Y, DISPLAY_W - 1, DIV_Y], fill="white")
```

# Sentence area

```
draw.rectangle([0, SENT_Y, DISPLAY_W - 1, PRED_Y - 1], fill="black")
```

if not words:

```
    draw.text((2, SENT_Y + 2), "No words yet", font=f_m, fill="white")
```

else:

```
    sentence = " ".join(words)
```

```
    lines = _wrap(draw, sentence, f_m, DISPLAY_W - 4)
```

```
    visible = max(1, SENT_H // LINE_H)
```

```
    off = min(scroll, max(0, len(lines) - visible))
```

```
    y = SENT_Y + 2
```

```
    for line in lines[off: off + visible]:
```

```
        draw.text((2, y), line, font=f_m, fill="white")
```

```
        y += LINE_H
```

# Prediction strip

```
draw.rectangle([0, PRED_Y, DISPLAY_W - 1, DISPLAY_H - 1], fill="black")
```

```
draw.line([0, PRED_Y, DISPLAY_W - 1, PRED_Y], fill="white")
```

if prediction:

```
word = prediction.upper()

tw  = _tw(draw, word, f_l)

tx  = max(0, (DISPLAY_W - tw) // 2)

draw.text((tx, PRED_Y + 1), word, font=f_l, fill="white")

else:

    draw.text((4, PRED_Y + 3), "Waiting...", font=f_s, fill="white")
```

```
def _draw_flash_ui(draw, text):

    f = font_medium or ImageFont.load_default()

    draw.rectangle([0, 0, DISPLAY_W - 1, DISPLAY_H - 1], fill="black")

    lines = _wrap(draw, text, f, DISPLAY_W - 6)

    total = len(lines) * LINE_H

    y  = max(2, (DISPLAY_H - total) // 2)

    for line in lines:

        draw.text((3, y), line, font=f, fill="white")

        y += LINE_H
```

```
def _draw_context_ui(draw, ctx):

    f_h = font_small or ImageFont.load_default()

    f_t = font_medium or ImageFont.load_default()

    draw.rectangle([0, 0, DISPLAY_W - 1, DISPLAY_H - 1], fill="black")

    draw.text((2, 1), "CONTEXT:", font=f_h, fill="white")

    draw.line([0, DIV_Y, DISPLAY_W - 1, DIV_Y], fill="white")

    lines = _wrap(draw, ctx, f_t, DISPLAY_W - 4)
```

```
y = DIV_Y + 3
```

```
for line in lines[:4]:
```

```
    draw.text((2, y), line, font=f_t, fill="white")
```

```
    y += LINE_H
```

```
# =====
```

```
# OLED DISPLAY THREAD
```

```
# =====
```

```
def oled_display_thread():
```

```
    if not OLED_AVAILABLE:
```

```
        print("[OLED] Not available — thread exiting")
```

```
        return
```

```
    try:
```

```
        serial = i2c(port=1, address=0x3C)
```

```
        device = sh1106(serial)
```

```
        print("[OLED] SH1106 ready ✓")
```

```
    except Exception as e:
```

```
        print(f"[OLED] Init failed: {e}")
```

```
        return
```

```
_load_fonts()
```

```
try:
```

```
    with canvas(device) as draw:
```

```
        f = font_medium or ImageFont.load_default()
```

```
draw.text((10, 20), "Sign Lang", font=f, fill="white")
```

```
draw.text((10, 34), "System Ready", font=f, fill="white")
```

```
except Exception:
```

```
    pass
```

```
time.sleep(1.5)
```

```
while True:
```

```
    oled_event.wait(timeout=0.3)
```

```
    oled_event.clear()
```

```
    with display_lock:
```

```
        prediction = oled_prediction
```

```
        words      = list(oled_words)
```

```
        context    = oled_context
```

```
        status     = oled_status
```

```
        flash_text = oled_flash_text
```

```
        flash_until = oled_flash_until
```

```
        scroll      = oled_scroll_offset
```

```
        is_rec     = oled_is_recording
```

```
    now = time.time()
```

```
    try:
```

```
        with canvas(device) as draw:
```

```
            if flash_text and now < flash_until:
```

```
                _draw_flash_ui(draw, flash_text)
```



```
        elif context and now < flash_until:
            _draw_context_ui(draw, context)
        else:
            _draw_normal_ui(draw, status, is_rec, words, scroll, prediction)
    except Exception as e:
        print(f"[OLED] Render error: {e}")

# =====
# OLED STATE SETTERS
# =====

def oled_set_prediction(word):
    global oled_prediction
    with display_lock:
        oled_prediction = word
    oled_event.set()

def oled_set_words(words):
    global oled_words
    with display_lock:
        oled_words = list(words)
    oled_event.set()

def oled_set_status(s):
    global oled_status
    with display_lock:
```

```
    oled_status = s
    oled_event.set()

def oled_set_recording(v):
    global oled_is_recording
    with display_lock:
        oled_is_recording = v
    oled_event.set()

def oled_flash(text, duration=4.0):
    global oled_flash_text, oled_flash_until
    with display_lock:
        oled_flash_text = text
        oled_flash_until = time.time() + duration
    oled_event.set()

def oled_show_context(text, duration=6.0):
    global oled_context, oled_flash_until
    with display_lock:
        oled_context = text
        oled_flash_until = time.time() + duration
    oled_event.set()

def oled_scroll(direction):
    global oled_scroll_offset
    with display_lock:
```

```
oled_scroll_offset = max(0, oled_scroll_offset + direction)

oled_event.set()
```

```
# =====
# WEBSOCKET CONNECTION
# =====
```

```
def connect_ws():
    print(f"Connecting to {WS_URL} ...")
    oled_set_status("CONNECTING")
    retry = 0
    while True:
        try:
            ws = websocket.create_connection(WS_URL, timeout=3)
            ws.settimeout(1)
            print("✓ Connected")
            oled_set_status("READY")
            return ws
        except (websocket.WebSocketException, socket.error) as e:
            retry += 1
            oled_set_status(f"RETRY {retry}")
            time.sleep(2)
```

```
# =====
# LANDMARK EXTRACTION
# =====
```

```
def extract_landmarks(results):
    if not results.multi_hand_landmarks:
        return None
    all_lm = [-1.0] * 128
    for idx, (hl, hi) in enumerate(
        zip(results.multi_hand_landmarks, results.multi_handedness)):
        if idx >= 2:
            break
        coords = []
        for lm in hl.landmark:
            coords.extend([float(lm.x), float(lm.y), float(lm.z)])
        label = 0.0 if hi.classification[0].label == 'Left' else 1.0
        start = idx * 64
        all_lm[start: start + 63] = coords
        all_lm[start + 63] = label
    return all_lm

# =====
# CAPTURE FRAME
# =====

def capture_frame_as_base64():
    with latest_frame_lock:
        if latest_frame is None:
            return None
```

```
        fc = latest_frame.copy()

        ok, buf = cv2.imencode('.jpg', fc, [cv2.IMWRITE_JPEG_QUALITY,
CAPTURE_JPEG_QUALITY])

        return base64.b64encode(buf.tobytes()).decode('utf-8') if ok else None

# =====

# AUDIO RECORDING

# =====

def _audio_cb(indata, frames, time_info, status):

    with audio_lock:

        if audio_recording:

            audio_recorded_chunks.append(indata.copy())

def record_audio_fixed(duration_seconds, ws):

    global audio_recording, audio_recorded_chunks

    if not AUDIO_AVAILABLE:

        _send_ws(ws, {'type': 'error', 'error': 'sounddevice not installed'})

        return

    duration_seconds = min(duration_seconds, AUDIO_MAX_SECONDS)

    with audio_lock:

        audio_recording = True

        audio_recorded_chunks = []

    try:

        _send_ws(ws, {'type': 'audio_status', 'status': 'recording',

            'duration': duration_seconds})

        with sd.InputStream(samplerate=AUDIO_SAMPLE_RATE, channels=AUDIO_CHANNELS,
```

```
        dtype='int16', callback=_audio_cb):

    sd.sleep(int(duration_seconds * 1000))

except Exception as e:

    _send_ws(ws, {'type': 'error', 'error': str(e)})

    with audio_lock:

        audio_recording = False

        oled_set_recording(False)

    return

with audio_lock:

    audio_recording = False

    chunks = list(audio_recorded_chunks)

oled_set_recording(False)

_send_audio_chunks(chunks, duration_seconds, ws)


def record_audio_manual(ws):

    global audio_recording, audio_recorded_chunks

    if not AUDIO_AVAILABLE:

        _send_ws(ws, {'type': 'error', 'error': 'sounddevice not installed'})

        return

    audio_stop_event.clear()

    with audio_lock:

        audio_recording = True

        audio_recorded_chunks = []

    try:

        _send_ws(ws, {'type': 'audio_status', 'status': 'recording', 'duration': 0})

        with sd.InputStream(samplerate=AUDIO_SAMPLE_RATE, channels=AUDIO_CHANNELS,
```

```
        dtype='int16', callback=_audio_cb):

    while not audio_stop_event.is_set():

        sd.sleep(100)

except Exception as e:

    _send_ws(ws, {'type': 'error', 'error': str(e)})

    with audio_lock:

        audio_recording = False

        oled_set_recording(False)

    return

with audio_lock:

    audio_recording = False

    chunks = list(audio_recorded_chunks)

    oled_set_recording(False)

    duration = len(chunks) * 1024 / AUDIO_SAMPLE_RATE

    _send_audio_chunks(chunks, duration, ws)


def _send_audio_chunks(chunks, duration, ws):

    if not chunks:

        _send_ws(ws, {'type': 'error', 'error': 'No audio recorded'})

        return

    audio_np = np.concatenate(chunks, axis=0)

    audio_b64 = base64.b64encode(audio_np.tobytes()).decode('utf-8')

    _send_ws(ws, {

        'type': 'audio', 'audio': audio_b64,

        'rate': AUDIO_SAMPLE_RATE, 'channels': AUDIO_CHANNELS,

        'duration': round(duration, 2), 'encoding': 'int16_pcm',
```

```
})
```

```
print("[AUDIO] Sent ✓")
```

```
# =====
```

```
# WS HELPER
```

```
# =====
```

```
def _send_ws(ws, payload):
```

```
    if ws is None:
```

```
        return
```

```
    try:
```

```
        ws.send(json.dumps(payload))
```

```
    except Exception as e:
```

```
        print(f"[WS] Send error: {e}")
```

```
# =====
```

```
# KEYPAD THREAD
```

```
# =====
```

```
def keypad_thread(ws_ref):
```

```
    if not GPIO_AVAILABLE:
```

```
        print("[KEYPAD] GPIO not available — disabled")
```

```
        return
```

```
    GPIO.setmode(GPIO.BCM)
```

```
    for r in ROWS:
```



```
GPIO.setup(r, GPIO.OUT)

GPIO.output(r, GPIO.HIGH)

for c in COLS:

    GPIO.setup(c, GPIO.IN, pull_up_down=GPIO.PUD_UP)


print("[KEYPAD] Ready ✓")

last_key = None

last_time = 0.0


while True:

    try:

        for i, r in enumerate(ROWS):

            GPIO.output(r, GPIO.LOW)

            for j, c in enumerate(COLS):

                if GPIO.input(c) == 0:

                    key = KEYS[i][j]

                    now = time.time()

                    if key != last_key or (now - last_time) > 0.4:

                        last_key = key

                        last_time = now

                        # Notify laptop of keypress for UI highlight

                        _send_ws(ws_ref[0], {

                            'type': 'keypad_press',

                            'key': key,

                            'action': KEY_ACTIONS.get(key, ""),

                        })
```

```
        _handle_key(key, ws_ref)

        time.sleep(0.05)

        GPIO.output(r, GPIO.HIGH)

        time.sleep(0.02)

    except Exception as e:

        print(f"[KEYPAD] Error: {e}")

        time.sleep(0.1)
```

```
def _handle_key(key, ws_ref):

    global oled_scroll_offset, oled_context
```

```
    ws    = ws_ref[0]

    action = KEY_ACTIONS.get(key)

    if not action:

        return
```

```
    print(f"[KEY] {key} → {action}")
```

```
# — Scroll —————
```

```
if action == 'scroll_up':
```

```
    oled_scroll(+1)
```

```
    return
```

```
if action == 'scroll_down':
```

```
    oled_scroll(-1)
```

```
    return
```

# — Preset 1-8 —————

```
if action.startswith('preset_'):
    slot = int(action.split('_')[1])
    with preset_lock:
        sentence = preset_cache.get(slot, "")
    if sentence:
        _send_ws(ws, {'command': 'trigger_preset', 'slot': slot, 'sentence': sentence})
        oled_set_status(f"P{slot} SENT")
        oled_flash(f"Preset {slot}:\n{sentence[:40]}", 4.0)
        print(f"[PRESET {slot}] {sentence}")
    else:
        oled_flash(f"Preset {slot}\nnot set", 2.0)
return
```

# — Add Word —————

```
if action == 'add_word':
    with display_lock:
        word = oled_prediction
    if word:
        _send_ws(ws, {'command': 'add_word', 'word': word})
        oled_set_status(f"+ {word.upper()}")
    else:
        oled_flash("No word detected", 2.0)
return
```

# — Delete Last —————

```
if action == 'delete_last':  
    _send_ws(ws, {'command': 'delete_last'})  
    oled_set_status("DELETED LAST")  
    return
```

# — Clear All —————

```
if action == 'clear_words':  
    _send_ws(ws, {'command': 'clear_words'})  
    oled_set_words([])  
    with display_lock:  
        oled_scroll_offset = 0  
    oled_set_status("CLEARED")  
    return
```

# — Send to Gemini —————

```
if action == 'send_to_ai':  
    with display_lock:  
        has_words = len(oled_words) > 0  
    if not has_words:  
        oled_flash("Add words first", 2.5)  
        return  
    _send_ws(ws, {'command': 'send_to_ai'})  
    oled_set_status("GEMINI...")  
    oled_flash("Sending to\nGemini...", 30.0)  
    return
```

# — OCR —————

```
if action == 'request_ocr':
    _send_ws(ws, {'command': 'request_ocr'})
    oled_set_status("OCR...")
    oled_flash("Capturing &\nReading...", 30.0)
    return
```

# — Toggle Recording —————

```
if action == 'toggle_audio':
    with audio_lock:
        rec = audio_recording
    if rec:
        audio_stop_event.set()
        _send_ws(ws, {'command': 'stop_audio'})
        oled_set_status("REC STOPPED")
        oled_set_recording(False)
    else:
        _send_ws(ws, {'command': 'start_audio'})
        oled_set_status("RECORDING")
        oled_set_recording(True)
    return
```

# =====

# INCOMING MESSAGE LISTENER

# =====

```
def listen_for_commands(ws_ref):  
    global oled_prediction, oled_context, oled_scroll_offset  
  
    print("[CMD] Listener started")  
    while True:  
        ws = ws_ref[0]  
        try:  
            raw = ws.recv()  
            if not raw:  
                continue  
  
            data = json.loads(raw)  
            msg_type = data.get('type')  
  
            # — Predicted word —————  
            if msg_type == 'show_word':  
                word = data.get('word', "")  
                with display_lock:  
                    oled_prediction = word  
                oled_event.set()  
                continue  
  
            # — Words list updated —————  
            if msg_type == 'words_updated':  
                oled_set_words(data.get('words', []))
```

```
oled_set_status("READY")
```

```
continue
```

```
# — Preset data from laptop —————
```

```
if msg_type == 'presets_updated':
```

```
    presets = data.get('presets', {})
```

```
    with preset_lock:
```

```
        for k, v in presets.items():
```

```
            try:
```

```
                preset_cache[int(k)] = v
```

```
            except (ValueError, KeyError):
```

```
                pass
```

```
    print(f"[PRESET] Cache updated: {preset_cache}")
```

```
    continue
```

```
# — Gemini generating —————
```

```
if msg_type == 'gemini_status':
```

```
    if data.get('status') == 'generating':
```

```
        oled_set_status("GEMINI...")
```

```
        oled_flash("Generating\nsentence...", 30.0)
```

```
    else:
```

```
        oled_set_status("READY")
```

```
    continue
```

```
# — Gemini sentence result —————
```

```
if msg_type == 'gemini_sentence':
```

```
sentence = data.get('sentence', "")
oled_set_status("GEMINI DONE")
oled_flash(sentence, 8.0)
continue
```

```
# — OCR result —————
```

```
if msg_type == 'ocr_result':
    text = data.get('text', "")
    if text:
        with display_lock:
            oled_context = text
            oled_scroll_offset = 0
            oled_set_status("OCR DONE")
            oled_show_context(text, 6.0)
            _send_ws(ws_ref[0], {'command': 'add_context', 'text': text})
        continue
```

```
# — Audio status —————
```

```
if msg_type == 'audio_status':
    s = data.get('status', "")
    if s == 'recording':
        oled_set_status("RECORDING")
        oled_set_recording(True)
    elif s == 'transcribing':
        oled_set_status("TRANSCRIBING")
    continue
```



```
# — Audio / Whisper result —————  
  
if msg_type == 'audio_result':  
    text = data.get('text', "")  
    oled_set_recording(False)  
    if text:  
        with display_lock:  
            oled_context = text  
            oled_scroll_offset = 0  
            oled_set_status("AUDIO DONE")  
            oled_show_context(text, 6.0)  
            _send_ws(ws_ref[0], {'command': 'add_context', 'text': text})  
        continue
```

```
# — Context updated —————  
  
if msg_type == 'context_updated':  
    ctx = data.get('context', "")  
    with display_lock:  
        oled_context = ctx  
    if not ctx:  
        oled_set_status("CTX CLEARED")  
        oled_event.set()  
    continue
```

```
# — Error —————  
  
if msg_type == 'error':
```

```
err = data.get('error', {})  
  
ctx_str = err.get('context', 'Error') if isinstance(err, dict) else 'Error'  
msg_str = err.get('error', str(err)) if isinstance(err, dict) else str(err)  
  
oled_set_status("ERROR")  
  
oled_flash(f"{ctx_str}:\n{msg_str}", 5.0)  
  
print(f"[ERR] {ctx_str}: {msg_str}")  
  
continue  
  
  
# — Commands from laptop —————  
  
if msg_type != 'command':  
    continue  
  
  
command = data.get('command', "")  
  
  
if command == 'capture_frame':  
    b64 = capture_frame_as_base64()  
    if b64:  
        _send_ws(ws, {  
            'type': 'frame', 'frame': b64,  
            'width': CAPTURE_WIDTH, 'height': CAPTURE_HEIGHT,  
            'timestamp': time.time(),  
        })  
    else:  
        _send_ws(ws, {'type': 'error', 'error': 'No frame available'})  
  
  
elif command == 'record_audio':
```

```
duration = data.get('params', {}).get('duration', 10)

with audio_lock:

    rec = audio_recording

if not rec:

    threading.Thread(target=record_audio_fixed,
                    args=(duration, ws), daemon=True).start()

else:

    _send_ws(ws, {'type': 'error', 'error': 'Already recording'})

elif command == 'start_audio':

    with audio_lock:

        rec = audio_recording

    if not rec:

        threading.Thread(target=record_audio_manual,
                        args=(ws,), daemon=True).start()

elif command == 'stop_audio':

    with audio_lock:

        rec = audio_recording

    if rec:

        audio_stop_event.set()

except websocket.WebSocketTimeoutException:

    continue

except (websocket.WebSocketConnectionClosedException, BrokenPipeError, OSError):

    time.sleep(0.5)
```

```
except json.JSONDecodeError as e:
```

```
    print(f"[CMD] Bad JSON: {e}")
```

```
except Exception as e:
```

```
    print(f"[CMD] Error: {e}")
```

```
    time.sleep(0.1)
```

```
# =====
```

```
# CV2 WINDOW
```

```
# =====
```

```
def calculate_fps():
```

```
    global frame_count, fps_val, last_fps_time
```

```
    frame_count += 1
```

```
    now = time.time()
```

```
    if now - last_fps_time >= 1.0:
```

```
        fps_val = frame_count / (now - last_fps_time)
```

```
        frame_count = 0
```

```
        last_fps_time = now
```

```
    return fps_val
```

```
def draw_info(image, hands_detected, data_sent):
```

```
    overlay = image.copy()
```

```
    cv2.rectangle(overlay, (10, 10), (270, 155), (0, 0, 0), -1)
```

```
    cv2.addWeighted(overlay, 0.6, image, 0.4, 0, image)
```

```
    y = 35
```

```
    cv2.putText(image, "Sign Language Sender", (20, y),
```

```
        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

    y += 25

    cv2.putText(image, f"Hands: {hands_detected}", (20, y),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                (0, 255, 0) if hands_detected > 0 else (0, 0, 255), 1)

    y += 25

    cv2.putText(image, f"FPS: {calculate_fps():.1f}", (20, y),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

    y += 25

    cv2.putText(image, "SENDING" if data_sent else "IDLE", (20, y),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                (0, 255, 0) if data_sent else (255, 255, 255), 1)

    y += 25

    with audio_lock:
        rec = audio_recording

    if rec:
        cv2.putText(image, "* REC AUDIO", (20, y),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)

    else:
        cv2.putText(image, "S=Capture Q=Quit", (20, y),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, (200, 200, 200), 1)

    cv2.putText(image, "Press 'q' to quit", (20, image.shape[0] - 20),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

# =====

# MAIN
```

```
# =====
```

```
def main():
```

```
    global last_send_time, last_heartbeat_time, latest_frame
```

```
    ws = connect_ws()
```

```
    ws_ref = [ws]
```

```
    ws_ref_holder[0] = ws
```

```
    threading.Thread(target=oled_display_thread, daemon=True).start()
```

```
    threading.Thread(target=keypad_thread, args=(ws_ref,), daemon=True).start()
```

```
    threading.Thread(target=listen_for_commands, args=(ws_ref,), daemon=True).start()
```

```
    cap = cv2.VideoCapture(0)
```

```
    if not cap.isOpened():
```

```
        print("ERROR: Cannot open camera!")
```

```
        return
```

```
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, CAPTURE_WIDTH)
```

```
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, CAPTURE_HEIGHT)
```

```
    cap.set(cv2.CAP_PROP_FPS, 30)
```

```
    print(f"Camera: {int(cap.get(3))}x{int(cap.get(4))}")
```

```
    print("Detection loop started\n")
```

```
    try:
```

```
        while cap.isOpened():
```

```
ok, image = cap.read()

if not ok:
    continue

now = time.time()

image = cv2.flip(image, 1)

with latest_frame_lock:
    latest_frame = image.copy()

rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
results = hands.process(rgb)

data_sent = False
hands_detected = 0

if results.multi_hand_landmarks:
    hands_detected = len(results.multi_hand_landmarks)
    if SHOW_LANDMARKS:
        for hl in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(image, hl, mp_hands.HAND_CONNECTIONS,
                                       landmark_spec, connection_spec)
    if (now - last_send_time) >= SEND_INTERVAL:
        lm = extract_landmarks(results)
        if lm and len(lm) == 128:
            try:
```

```
        ws.send(json.dumps({'landmarks': lm, 'timestamp': now,
                             'hands': hands_detected}))

        data_sent      = True

        last_send_time  = now

        last_heartbeat_time = now

        ws_ref[0]       = ws

        ws_ref_holder[0] = ws

    except (websocket.WebSocketConnectionClosedException,
            socket.error, BrokenPipeError):

        ws = connect_ws()

        ws_ref[0] = ws

        ws_ref_holder[0] = ws

elif (now - last_heartbeat_time) >= HEARTBEAT_INTERVAL:

    try:

        ws.send(json.dumps({'type': 'ping', 'status': 'idle', 'timestamp': now}))

        last_heartbeat_time = now

    except Exception:

        pass

draw_info(image, hands_detected, data_sent)

cv2.imshow('Sign Language (Pi)', cv2.resize(image, (854, 480)))

key = cv2.waitKey(1) & 0xFF

if key == ord('s'):

    b64 = capture_frame_as_base64()
```



```
        if b64:
            try:
                ws.send(json.dumps({'type': 'frame', 'frame': b64,
                                    'width': CAPTURE_WIDTH, 'height': CAPTURE_HEIGHT,
                                    'timestamp': now}))

            except Exception as e:
                print(f"[MANUAL] {e}")

        elif key == ord('q'):
            break

except KeyboardInterrupt:
    pass

except Exception as e:
    import traceback
    traceback.print_exc()

finally:
    cap.release()
    cv2.destroyAllWindows()

    if GPIO_AVAILABLE:
        GPIO.cleanup()

    try:
        ws.close()

    except Exception:
        pass

    print("Shutdown complete")
```

```
if __name__ == "__main__":  
    main()
```

---